ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №1

«Організація професійного Python-проєкту та Git»

з курсу «Професійний Python»
Варіант №3 — «**Інтернет-магазин** »

Виконав(ла): Вересюк Ірина

Група: ФЕП-32

Перевірив(ла):Куньо І.М.

Дата: 08.10.26

Львів — 2026

1.Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

2. Індивідуальний варіант і постановка задачі

Варіант №3 — «Інтернет-магазин». Для кожного товару зберігати назву, категорію, ціну та кількість. Реалізувати додавання товару, обчислення вартості залишків, пошук товару, фільтрацію за категорією та визначення найдорожчого товару.

Проєкт повинен мати src-layout, щонайменше три Python modules, data model, окремий module з business logic, main.py, type hints, README.md, .gitignore, pyproject.toml і Git repository.

3. Структура проєкту

```
eshop_manager/
├── pyproject.toml
├── README.md
├── .gitignore
├── report/
│   └── Вересюк_Ірина_Лаб01_V3.pdf
├── src/
│   └── eshop_manager/
│       ├── __init__.py
│       ├── main.py
│       ├── models.py
│       └── services.py
└── tests/
```

Каталог src/eshop_manager містить пакет застосунку. models.py описує модель даних Product, services.py містить business logic, main.py є точкою входу з реалізацією меню користувача.

4. Опис створених модулів

4.1. models.py

Для представлення товару використано dataclass Product із полями name, category, price та quantity. Метод __post_init__ перевіряє коректність даних за допомогою винятку InvalidProductDataError, а властивість total_cost обчислює сумарну вартість залишку одиниці товару на складі.

```
from dataclasses import dataclass


class InvalidProductDataError(Exception):
    """Виняток для некоректних даних товару."""
    pass


@dataclass
class Product:
    """Модель товару інтернет-магазину."""
    name: str
    category: str
    price: float
    quantity: int


    def __post_init__(self) -> None:
        if self.price < 0:
            raise InvalidProductDataError("Ціна не може бути від'ємною.")
        if self.quantity < 0:
            raise InvalidProductDataError("Кількість не може бути від'ємною.")
```

```
    @property
    def total_cost(self) -> float:
        """Вартість залишку поточного товару на складі."""
        return self.price * self.quantity
```

Модуль відповідає за шар даних проєкту. Використання декоратора @dataclass автоматизує генерацію методів __init__, __repr__ та __eq__. Впроваджено власний клас винятку InvalidProductDataError та метод валідації __post_init__, який унеможливлює створення товару з від'ємною ціною чи кількістю. Властивість total_cost інкапсулює розрахунок загальної вартості позиції на складі.

4.2. services.py

У модулі реалізовано функції calculate_inventory_value(), find_product_by_name(), filter_by_category() та find_most_expensive_product(). Business logic не залежить від консольного інтерфейсу введення/виведення.

```
from eshop_manager.models import Product


def calculate_inventory_value(products: list[Product]) -> float:
    """Обчислення загальної вартості всіх залишків товарів."""
    return sum(product.total_cost for product in products)


def find_product_by_name(products: list[Product], name: str) -> Product | None:
    """Пошук товару за назвою (без урахування регістру)."""
    search_name = name.strip().lower()
    for product in products:
        if product.name.lower() == search_name:
            return product
    return None
```

```
def filter_by_category(products: list[Product], category: str) -> list[Product]:
    """Фільтрація списку товарів за категорією."""
    filter_category = category.strip().lower()
    return [
        product
        for product in products
        if product.category.lower() == filter_category
    ]


def find_most_expensive_product(products: list[Product]) -> Product | None:
    """Визначення найдорожчого товару за одиницю вартості."""
    if not products:
        return None
    return max(products, key=lambda product: product.price)
```

Модуль концентрує в собі всю бізнес-логіку обробки даних предметної області. Функції реалізовано як «чисті» операції, що приймають колекцію об'єктів і повертають результат або відфільтровані дані без звернення до print() чи input(). Це забезпечує легке юніт-тестування та ізоляцію алгоритмів від інтерфейсу користувача.

4.3. main.py

Модуль main.py створює початкові дані, містить функції консольного інтерфейсу з валідацією вводу та меню користувача. Конструкція if __name__ == "__main__": відокремлює точку входу від функцій програми.

```
from eshop_manager.models import InvalidProductDataError, Product
from eshop_manager.services import (
    calculate_inventory_value,
    filter_by_category,
    find_most_expensive_product,
    find_product_by_name,
```

```
)


def create_demo_products() -> list[Product]:
    """Створення початкових демонстраційних даних."""
    return [
        Product(name="Ноутбук Lenovo", category="Електроніка", price=28500.0, quantity=5),
        Product(name="Смартфон Samsung", category="Електроніка", price=19999.0, quantity=8),
        Product(name="Кавоварка Philips", category="Побутова техніка", price=6400.0, quantity=3),
        Product(name="Мишка бездротова", category="Аксесуари", price=850.0, quantity=15),
    ]


def print_products(products: list[Product]) -> None:
    """Форматоване виведення списку товарів."""
    if not products:
        print("\nСписок товарів порожній.")
        return

    print("\n" + "=" * 65)
    print(f"{'Назва':25} | {'Категорія':18} | {'Ціна (грн)':10} | {'К-сть':5}")
    print("-" * 65)
    for p in products:
        print(f"{p.name:25} | {p.category:18} | {p.price:10.2f} | {p.quantity:5d}")
    print("=" * 65)


def add_product_cli(products: list[Product]) -> None:
```

```
    """Додавання товару через консоль із валідацією."""
    name = input("Введіть назву товару: ").strip()
    if not name:
        print("Помилка: назва не може бути порожньою.")
        return

    category = input("Введіть категорію: ").strip()
    if not category:
        print("Помилка: категорія не може бути порожньою.")
        return

    try:
        price = float(input("Введіть ціну: "))
        quantity = int(input("Введіть кількість: "))
        new_product = Product(name=name, category=category, price=price,
quantity=quantity)
        products.append(new_product)
        print(f"Товар '{name}' успішно додано!")
    except ValueError:
        print("Помилка: ціна та кількість мають бути числовими значеннями.")
    except InvalidProductDataError as err:
        print(f"Помилка створення товару: {err}")

def print_menu() -> None:
    print("\n--- Меню Інтернет-Магазину ---")
    print("1. Показати всі товари")
    print("2. Додати новий товар")
    print("3. Обчислити загальну вартість залишків")
```

```
    print("4. Знайти товар за назвою")
    print("5. Фільтрувати за категорією")
    print("6. Показати найдорожчий товар")
    print("0. Вихід")

def main() -> None:
    products = create_demo_products()

    while True:
        print_menu()
        choice = input("Оберіть дію (0-6): ").strip()

        if choice == "1":
            print_products(products)
        elif choice == "2":
            add_product_cli(products)
        elif choice == "3":
            total_val = calculate_inventory_value(products)
            print(f"\nЗагальна вартість усіх залишків на складі: {total_val:.2f}
грн")
        elif choice == "4":
            name = input("Введіть назву для пошуку: ")
            product = find_product_by_name(products, name)
            if product:
                print(f"Знайдено: {product.name} ({product.category}) —
{product.price:.2f} грн, {product.quantity} шт.")
            else:
                print("Товар з такою назвою не знайдено.")
```

```
    elif choice == "5":
        cat = input("Введіть назву категорії: ")
        filtered = filter_by_category(products, cat)
        print_products(filtered)
    elif choice == "6":
        expensive = find_most_expensive_product(products)
        if expensive:
            print(f"Найдорожчий товар: {expensive.name} — 
{expensive.price:.2f} грн")
        else:
            print("Список товарів порожній.")
    elif choice == "0":
        print("Роботу завершено. До побачення!")
        break
    else:
        print("Невідома команда. Спробуйте ще раз.")


if __name__ == "__main__":
    main()
```

Модуль слугує точкою входу в застосунок та реалізує шар взаємодії з користувачем через циклічне меню. Він ініціалізує початковий набір даних, приймає команди, валідує вхідні типи даних від користувача, перехоплює винятки InvalidProductDataError та ValueError, передає дані у функції сервісів і виводить відформатовані звіти.

5. Файл pyproject.toml

Файл задає build system setuptools, метадані проєкту, версію Python 3.11+, console script та налаштування каталогу пошуку пакетів у src.

```
[build-system]
requires = ["setuptools>=70"]
```

build-backend = "setuptools.build_meta"

[project]

name = "eshop-manager"

version = "0.1.0"

description = "E-shop management console system"

requires-python = ">=3.11"

[project.scripts]

eshop-manager = "eshop_manager.main:main"

[tool.setuptools.packages.find]

where = ["src"]

Файл виступає центральним стандартизованим декларативним конфігуратором проєкту згідно з PEP 518 та PEP 621. Секція [build-system] вказує рушій збирання, секція [project] фіксує ідентифікатори та сумісність із версією Python, [project.scripts] реєструє глобальну консольну команду запуску eshop-manager, а секція [tool.setuptools.packages.find] забезпечує правильне знаходження пакета в каталозі src/.

6. Файл .gitignore

З Git виключено virtual environment, кеші, IDE configuration, build artifacts і .env.

.venv/

__pycache__/

*.pyc

.pytest_cache/

.idea/

.vscode/

dist/

build/

*.egg-info/

.env

Даний конфігураційний файл фільтрує робочий простір репозиторію від службових, тимчасових та автогенерованих файлів. Він запобігає потраплянню у відстеження Git директорій віртуального середовища .venv/, байт-коду __pycache__/, кешів середовищ розробки (.vscode/, .idea/) та збіркових директорій *.egg-info/.

7. Фрагмент README.md

E-Shop Manager

Навчальний консольний проєкт для дисципліни «Професійний Python» (Лабораторна робота №1, Варіант 3).

Опис

Система обліку та управління товарами інтернет-магазину. Дозволяє додавати товари, шукати, фільтрувати за категоріями, знаходити найдорожчий товар та розраховувати сумарну вартість залишків.

Вимоги

- Python 3.11+

Встановлення

1. Створення та активація віртуального середовища:

```bash

python -m venv .venv

source .venv/bin/activate  # Для Linux/macOS

# .venv\Scripts\activate   # Для Windows

Розділ демонструє головний документ первинного ознайомлення з проєктом. У ньому описано суть навчального проєкту, мінімальні вимоги
```

до середовища виконання та детальні покрокові інструкції для розгортання віртуального середовища, встановлення пакета в editable mode та команд його запуску.

8. Запуск та результат роботи програми

(.venv) PS D:\Робочий стіл\проф. python\eshop_manager> eshop-manager

--- Меню Інтернет-Магазину ---

1. Показати всі товари

2. Додати новий товар

3. Обчислити загальну вартість залишків

4. Знайти товар за назвою

5. Фільтрувати за категорією

6. Показати найдорожчий товар

0. Вихід

Оберіть дію (0-6): 1

==========

Назва | Категорія | Ціна (грн) | К-сть

Ноутбук Lenovo | Електроніка | 28500.00 | 5

Смартфон Samsung | Електроніка | 19999.00 | 8

Кавоварка Philips | Побутова техніка | 6400.00 | 3

Мишка бездротова | Аксесуари | 850.00 | 15

==========

--- Меню Інтернет-Магазину ---

1. Показати всі товари

2. Додати новий товар

3. Обчислити загальну вартість залишків

4. Знайти товар за назвою

5. Фільтрувати за категорією

6. Показати найдорожчий товар

0. Вихід

Оберіть дію (0-6): 3

Загальна вартість усіх залишків на складі: 334442.00 грн

--- Меню Інтернет-Магазину ---

1. Показати всі товари

2. Додати новий товар

3. Обчислити загальну вартість залишків

4. Знайти товар за назвою

5. Фільтрувати за категорією

6. Показати найдорожчий товар

0. Вихід

Оберіть дію (0-6):

Розділ демонструє реальний вивід консолі під час роботи програми: ініціалізація та відображення головного меню, табличне форматування початкового списку товарів (дія 1), а також точний розрахунок сумарної вартості всіх залишків товарів на складі за допомогою сервісної функції (дія 3)

10. Робота з Git

У проєкті створено три логічні commits.

Коміт 1: "Create project structure"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop_manager (master)

$ git add .gitignore pyproject.toml src/eshop_manager/__init__.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop_manager (master)

$ git commit -m "Create project structure"

[master (root-commit) 96d4d67] Create project structure

3 files changed, 28 insertions(+)

create mode 100644 .gitignore

create mode 100644 pyproject.toml

create mode 100644 src/eshop_manager/__init__.py

Ініціалізовано репозиторій та створено базовий каркас проєкту за схемою src-layout. У файлі .gitignore налаштовано виключення для середовища .venv, байт-коду та кешів. Конфігураційний файл pyproject.toml визначає метадані застосунку, систему збирання setuptools і консольну команду запуску, а файл __init__.py перетворює каталог на Python-пакет.

Коміт 2: "Add product model and business logic"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop_manager (master)

$ git add src/eshop_manager/models.py src/eshop_manager/services.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop_manager (master)

$ git commit -m "Add product model and business logic"

[master 6e6f9a8] Add product model and business logic

2 files changed, 58 insertions(+)

create mode 100644 src/eshop_manager/models.py

create mode 100644 src/eshop_manager/services.py

Створено модулі сутностей та алгоритмів обробки даних магазину. У файлі models.py реалізовано датаклас Product з полями назви, категорії,

ціни, кількості, властивістю розрахунку вартості та валідацією значень через власний виняток. Модуль services.py містить чисту бізнес-логіку: підрахунок сумарної вартості залишків, пошук за назвою, фільтрацію за категорією та визначення найдорожчого товару.

Коміт 3: "Add application entry point and documentation"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop_manager (master)

$ git add src/eshop_manager/main.py README.md

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop_manager (master)

$ git commit -m "Add application entry point and documentation"

[master 2eabbc3] Add application entry point and documentation

2 files changed, 124 insertions(+)

create mode 100644 README.md

create mode 100644 src/eshop_manager/main.py

Реалізовано взаємодію з користувачем та базову документацію. Модуль main.py слугує точкою входу з конструкцією if __name__ == "__main__": і забезпечує роботу інтерактивного консольного меню, виведення таблиці товарів та обробку введених користувачем даних. Документ README.md описує призначення проєкту, системні вимоги та кроки для створення оточення, встановлення пакета і його запуску.

10.1. Результат git status

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop_manager (main)

$ git status

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Ця команда показує поточний стан робочого дерева (working tree) та індексу (staging area) репозиторію. Повідомлення On branch main свідчить про те, що робота ведеться в основній гілці проєкту. Рядок nothing to commit, working tree clean підтверджує, що всі файли успішно збережені в коміти, не відстежуваних чи змінених файлів немає, а каталог .venv та системні кеші правильно ігноруються завдяки налаштованому файлу

10.2. Результат git log –oneline

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop_manager (main)

$ git log --oneline

2eabbc3 (**HEAD** -> **main**, **origin/main**) Add application entry point and documentation

6e6f9a8 Add product model and business logic

96d4d67 Create project structure

Команда git log --oneline виводить скорочену хронологічну історію комітів у компактному однорядковому форматі: 7-значний скорочений хеш та коментар. Згенерований вивід демонструє виконання вимог лабораторної роботи щодо поетапної розробки проєкту. Історія чітко показує послідовність із трьох логічних етапів: створення базової структури (96d4d67), реалізація моделі даних та бізнес-логіки (6e6f9a8) і фінальне додавання точки входу з документацією (2eabbc3).

12. Висновок

У лабораторній роботі створено структурований Python-проєкт за src-layout для системи управління товарами інтернет-магазину. Реалізовано data model Product, окремий модуль business logic, точку входу main.py, type hints, валідацію вводу та обробку винятків. Підготовлено pyproject.toml, .gitignore і README.md, а також сформовано логічну Git-історію з трьох commits. Отримано практичні навички організації професійного Python-проєкту, роботи з virtual environment і Git.